CASE STUDY

The Multipath TCP Attack Surface: A 2026 Case Study

A TeamStarWolf case study. Published 3 November 2026; every source checked as of that date.

Executive summary

Multipath TCP (MPTCP) is a protocol extension that lets a single TCP connection run over several network paths at once, for throughput and for failover. The Linux kernel has shipped it since version 5.6 in 2020, and on most modern distributions the `net.mptcp.enabled` switch defaults to on. Almost nothing uses it. An application only gets MPTCP if it deliberately asks for it when it opens a socket, and the overwhelming majority of servers run software that asks for ordinary TCP. The result is a feature that is present and switched on across a large part of the Linux fleet while delivering value on a tiny fraction of it.

That gap between enabled and used is the subject of this study, because it is an attack surface. MPTCP is a large, relatively new body of kernel networking code, and new networking code written in a memory-unsafe language produces memory-safety bugs on a schedule. The kernel has fixed a steady run of MPTCP use-after-frees, memory leaks and resource-exhaustion flaws since the feature landed. For a host that never uses MPTCP, every one of those is pure downside: risk with no offsetting benefit. Worse, MPTCP can turn a bug that is elsewhere a mere crash into something far more serious, because its connection-management machinery hands an attacker more control over kernel memory. That is exactly what happened with CVE-2026-98432, where the presence of an MPTCP listener was the difference between a denial of service and a reported remote-root exploit.

The management problem is that none of this shows up in an ordinary vulnerability scan. Whether MPTCP is compiled in, whether it is enabled, and whether anything is actually using it are three separate questions, and a severity score answers none of them. This study explains what MPTCP is, why its mere presence raises the stakes on kernel memory bugs, how to check your own exposure across those three questions, why you would want to, and how to reduce the surface safely on the hosts that do not need it.

Evidence grade: the protocol behaviour and default state are **Confirmed** from the kernel's own documentation; the memory-safety flaws are **Confirmed** fixed vulnerabilities from kernel and vendor advisories; the remote-code-execution amplification on CVE-2026-98432 is **Reported** (single-source, unreproduced), as graded in the companion study.

Key facts at a glance

Field	Value
What it is	Multipath TCP, a TCP extension carrying one connection over multiple paths (subflows), standardized in RFC 8684 (v1), earlier RFC 6824 (v0)
In Linux since	Kernel 5.6 (2020); actively developed since
Default state	`net.mptcp.enabled` is a per-namespace boolean that defaults to 1 (MPTCP sockets can be created)
How an app uses it	The application must open the socket with `IPPROTO_MPTCP` (protocol number 262) instead of `IPPROTO_TCP` ; ordinary TCP code never touches MPTCP
Can be forced on	Yes, without changing the application, via `mptcpize` (LD_PRELOAD), `mptcpify` (eBPF) or SystemTAP
Security relevance	A large, newer kernel code path; a recurring source of memory-safety CVEs; it can upgrade the impact of a reachable kernel bug
The amplifier case	CVE-2026-98432 (CVSS 9.8): an MPTCP listener turned an IPv4 TCP use-after-free from a crash into a reported remote-root exploit
Safe reduction	On hosts that do not use MPTCP, set `net.mptcp.enabled=0` after confirming nothing requests it
What disabling does not do	It removes the MPTCP code path from reach; it does not patch an underlying plain-TCP bug, so it reduces exposure rather than replacing patching

Key findings

- **MPTCP is on by default and used by almost nothing.** The kernel documentation states that `net.mptcp.enabled` defaults to 1, so MPTCP sockets can be created on a stock modern kernel. But MPTCP is opt-in: an application receives it only by creating its socket with `IPPROTO_MPTCP` , and standard server software requests ordinary TCP. A host can therefore have MPTCP enabled for years without a single MPTCP connection.
- **Enabled is not the same as in use, and neither is the same as exposed.** Three questions have to be answered separately: is MPTCP compiled into this kernel, is it enabled by the sysctl, and is any process

actually listening or connecting with it. A vulnerability scanner that reports a kernel version answers none of them, and the worst-case risk depends on all three.

- **MPTCP is a recurring memory-safety surface.** Since the feature matured the kernel has fixed a series of MPTCP flaws, including use-after-free, memory leak and unbounded-memory-growth classes. Several carry high base scores, and for more than one the documented workaround is simply to disable MPTCP. For a host that never uses the feature, each is risk carried for no return.
- **MPTCP can amplify a bug from crash to compromise.** In CVE-2026-98432 the underlying flaw was an IPv4 TCP use-after-free. On a plain-TCP host the reachable outcome was a crash or memory corruption; on a host with an MPTCP listener it was a reported unauthenticated remote-root exploit, because MPTCP's machinery gives an attacker more control over kernel allocations. The feature did not cause the bug, but it changed what the bug was worth.
- **The exposure is invisible to normal inventory and cheap to remove.** Checking the three questions takes a few standard commands, and on a host that does not use MPTCP the fix is one sysctl setting. Because the control is a standing one, it also pre-reduces the blast radius of the next MPTCP CVE, before that CVE is disclosed.

Top recommendations

1. Inventory MPTCP across three states, not one. For every Linux host, record whether MPTCP is compiled in, whether `net.mptcp.enabled` is 1, and whether any socket is actually using it. Treat the combination, not the kernel version alone, as the exposure.
2. Disable MPTCP where nothing uses it. On hosts with no MPTCP sockets and no force-enable wrapper, set `net.mptcp.enabled=0` and persist it. This is ordinary attack-surface reduction: removing a protocol that is present but idle.
3. Confirm non-use before disabling. Disabling MPTCP makes a socket request for it fail outright, so a service that is hard-wired to MPTCP, or wrapped with `mptcpize`, will break. Check for listeners and wrappers first, and stage the change.
4. Prioritize the hosts that do use MPTCP. Where MPTCP is genuinely in use (some mobile-facing edge, multipath-aware services, specific telecom and research workloads), you cannot simply switch it off, so those hosts need the kernel patched promptly and watched more closely. They are also the hosts where an MPTCP bug reaches its worst case.
5. Treat MPTCP-disabled as a baseline control, not a one-off. Fold the setting into your hardening baseline and configuration management so new hosts inherit it, and so the next MPTCP CVE finds most of the fleet already out of reach.

Scope and method

This study is about MPTCP as an exposure to manage, not about a single incident. It draws the protocol's behaviour and default state from the Linux kernel's own documentation, the memory-safety history from kernel and vendor advisories, and the amplification example from the companion study on CVE-2026-98432. It is deliberately vendor-neutral: the checks and the mitigation use only standard kernel interfaces and common userspace tools, so they apply on any Linux host regardless of the management tooling around it.

Every claim carries one of three grades. Where advisory details differ between the authoritative source (the kernel stable tree or a distribution vendor) and third-party aggregators, the authoritative source governs and the study says so.

Grade	Meaning
Confirmed	The kernel project, a distribution vendor, or a government advisory confirms the behaviour, the vulnerability, or the fix
Reported	A single source's account, not independently corroborated or reproduced
Demonstrated	A research reproduction available for others to check

Part 1: what MPTCP is, and why it is almost always idle

The protocol

Multipath TCP is an extension to TCP that lets one logical connection use several network paths at the same time, each path carried as a subflow (**Confirmed**: RFC 8684, the kernel documentation). The motivating cases are real but specific: a phone that holds a connection together while moving between Wi-Fi and cellular, a server that aggregates bandwidth across two links, a service that survives the loss of one path by shifting traffic to another. MPTCP negotiates these capabilities in the TCP options of the handshake, and if the far end or a middlebox on the path does not understand them, the connection quietly continues as ordinary single-path TCP.

Linux has carried a mainline MPTCP implementation since kernel 5.6 in 2020, and it has grown steadily since, with an in-kernel path manager, packet schedulers, and a set of sysctl controls. It is a substantial and still-evolving body of networking code.

Why it is opt-in, and therefore usually unused

The decisive fact for exposure is that MPTCP is opt-in at the application level (**Confirmed**: kernel documentation). A program gets MPTCP only by creating its socket with the MPTCP protocol instead of TCP:

An application that creates an ordinary `IPPROTO_TCP` socket, which is almost all of them, never uses MPTCP no matter what the system setting is. The kernel documentation spells out the failure modes when a program does ask for MPTCP and cannot have it: on kernels before 5.6 the call fails with `EINVAL` ; on later kernels `EPROTONOSUPPORT` means MPTCP was not compiled into this kernel, and `ENOPROTOOPT` means it was compiled but turned off with `net.mptcp.enabled` .

This opt-in design is also why enabling MPTCP by default costs almost nothing in performance, and why vendors were comfortable shipping it on. The kernel notes that an MPTCP listen socket still produces plain accepted TCP sockets when a client's connection request does not ask for MPTCP. So a default-on MPTCP stack sits dormant behind the ordinary TCP traffic of ordinary software, waiting for a request that, on a typical server, never comes.

The default switch

The relevant control is a per-namespace sysctl (**Confirmed**: kernel `mptcp-sysctl` documentation):

Sysctl	Type	Default	Meaning
`net.mptcp.enabled`	boolean	1	Controls whether MPTCP sockets can be created; sockets can be created when the value is 1
`net.mptcp.add_addr_timeout`	integer (s)	120	Timeout before an ADD_ADDR control message is resent
`net.mptcp.checksum_enabled`	boolean	0	Enables the DSS checksum when nonzero
`net.mptcp.allow_join_initial_addr_port`	boolean	1	Allows peers to send join requests to the initial subflow's address and port
`net.mptcp.stale_loss_cnt`	integer	4	Idle retransmission intervals before a subflow is treated as stale
`net.mptcp.blackhole_timeout`	integer (s)	3600	Initial period MPTCP is disabled for after a firewall blackhole is detected

The one that matters for exposure is the first. Because it is per-namespace, its value in the host's root namespace does not automatically govern every container or network namespace on the box, which is a point the mitigation section returns to.

The force-enable paths

One nuance complicates the clean "apps request TCP, so MPTCP is idle" picture. The kernel documentation records that applications can be forced to use MPTCP without any code change, using `mptcpize` (which works through `LD_PRELOAD`), `mptcpify` (which uses eBPF), or SystemTAP (**Confirmed**). An operator may have wrapped a service this way on purpose, to get MPTCP's benefits under software that does not natively request it. The practical consequence is that "the application source requests TCP" is not sufficient proof that a host is idle on MPTCP: a `mptcpize` -wrapped service unit or an eBPF program can put MPTCP into use invisibly to anyone reading the application code. An exposure check has to look at the running sockets, not only at the software.

Part 2: why a dormant feature is an attack surface

New kernel code is a bug supply

The security argument does not depend on any single flaw. MPTCP is a large, comparatively young, and actively changing part of the kernel's network stack, written in C. Code with those properties produces memory-safety defects as a matter of course, and the kernel fixes them continuously. For a host that uses MPTCP, those fixes are the cost of a feature it relies on. For a host that does not, they are a stream of

vulnerabilities in code that is switched on but doing nothing useful.

The recurring flaws

The kernel's own advisory history bears this out. A representative set of MPTCP memory-safety and resource flaws, graded against their authoritative sources:

Identifier	Class	Impact as stated by the authoritative source	Grade
CVE-2026-31669	Use-after-free in IPv6 subflow child sockets	Red Hat: "at least a denial of service via kernel crash," a "memory corruption class issue that may allow confidentiality or integrity impact in a worst case"; reachable via MPTCP over IPv6 but with "no known attack scenario" and "no confirmed practical trigger scenario under default configurations"; Red Hat CVSS 7.5 (cve.org lists 9.8)	Confirmed
CVE-2026-80588	Unbounded forward-memory growth on the receive path	Kernel stable: memory can grow without bound when the receive path drops packets before queueing them, able to exhaust memory and crash the kernel; fixed in stable branches	Confirmed
CVE-2025-40258	Race-condition use-after-free	One advisory rates it CVSS 9.8 while also stating it is "very unlikely to be reachable from the internet," a split that illustrates why the base score alone misleads	Confirmed (reachability caveated)
CVE-2022-50071	Subflow socket resource leak when an eBPF program blocks socket creation	Can lead to memory exhaustion; the documented workaround is to disable MPTCP with `net.mptcp.enabled=0`	Confirmed
CVE-2021-47152	Data-stream corruption via page-fragment allocation	Could corrupt the data stream where other protocols such as plain TCP allocate page fragments	Confirmed

Two things stand out. First, the list is not exhaustive and the classes repeat: use-after-free, leak, exhaustion, corruption, the ordinary memory-safety failure modes of C networking code. Several of these advisories name

disabling MPTCP as the mitigation, which is the clearest possible signal that the feature is treated as optional. Second, aggregator sources disagree on details such as attack vector and severity (CVE-2026-31669 alone carries a Red Hat 7.5 and a cve.org 9.8), so the responsible reading keys on the kernel and vendor advisories and on your own reachability, not on the loudest number.

The amplifier

The sharpest reason to care about MPTCP exposure is not an MPTCP bug at all. It is the way MPTCP changes the value of a bug elsewhere in the stack. CVE-2026-98432 (CVSS 9.8) was a use-after-free in the IPv4 TCP connection-setup path, not in MPTCP code. Yet its impact split on MPTCP:

- On a host without an MPTCP listener, the reachable outcome on plain IPv4 TCP was a kernel crash, memory disclosure, or memory corruption, which is a denial of service and a possible stepping stone.
- On a host with an MPTCP listener, researchers reported a full unauthenticated remote-code-execution chain, because MPTCP's extra connection-management machinery gave the attacker more control over kernel allocations, which is what turns a use-after-free into reliable code execution.

The remote-code-execution demonstration is graded **Reported** in the companion study, resting on a single lab account that was not independently reproduced. But the structural point stands on the Confirmed vulnerability alone: the same bug was worth a crash on one host and a root shell on another, and the only difference was a protocol feature that the second host was not using for anything. The full account of that flaw, its two-line fix, and the AI-written exploit claim is in the CVE-2026-98432 case study. Here it serves as the clearest evidence that MPTCP exposure is not an abstract concern: a dormant feature decided whether a kernel bug was an outage or a breach.

Part 3: how to check your exposure, and why

The exposure question is not "is MPTCP enabled." It is three questions, and you want the answer to each because they gate different decisions: whether there is any surface at all, whether it is reachable, and whether you can safely remove it.

Question 1: is MPTCP compiled into this kernel?

If the kernel was built without `CONFIG_MPTCP` , there is no MPTCP surface to manage and nothing to disable. Check the build config:

`CONFIG_MPTCP=y` means it is built in. If the config is not exposed, the enabled check below answers the same question indirectly: the sysctl file only exists when MPTCP is compiled in.

Question 2: is MPTCP enabled?

This is the quick, universal check, and the one most people mean when they ask about MPTCP:

A `1` means MPTCP sockets can be created, which is the default on modern kernels. A `0` means it is turned off. If the file does not exist, MPTCP is not compiled in and you are done. The portable form that does not error when the path is absent:

Why you want this: it tells you, across the whole fleet, how much of your estate carries the surface at all. On a typical modern Linux fleet the answer will be "most of it," which is precisely why the next question matters.

Question 3: is anything actually using it?

This is the question that separates a real exposure from a theoretical one, and the one normal inventory never asks. Modern socket tooling can enumerate MPTCP sockets directly:

Empty output means that, at this moment, nothing on the host is using MPTCP, even though it is enabled. A LISTEN row means a service is accepting MPTCP connections and the host is genuinely in use. Because sockets come and go, treat a single empty result as a snapshot, not a verdict: sample across a business cycle, or re-check on a schedule, before concluding a host is idle.

Then look for the force-enable paths from Part 1, which can put MPTCP into use without any MPTCP-aware application:

Why you want this: the combination of enabled and in use, on a vulnerable kernel, is the worst-case cell. Enabled but idle is removable surface. Not enabled, or not compiled, is no surface. You cannot make that call from a kernel version, and you cannot make it from the enabled flag alone.

Putting the three together

The exposure of a host is the row it lands on:

Compiled in	Enabled	In use	What it means
No	n/a	n/a	No MPTCP surface; nothing to do
Yes	No	No	Already reduced; keep it that way
Yes	Yes	No	Removable surface: disable after confirming non-use (the common case)
Yes	Yes	Yes	Genuine use: cannot disable; patch promptly and watch; worst case for any MPTCP-reachable bug

Part 4: reducing the surface safely

The change

On a host that is enabled but idle, the reduction is one setting. Runtime, taking effect immediately but not surviving a reboot:

Persisted across reboots:

The gate: confirm non-use first

Disabling MPTCP is safe only on hosts that do not use it, and the failure mode if you get that wrong is not subtle. When MPTCP is disabled, a socket created with `IPPROTO_MPTCP` fails with `ENOPROTOOPT` . This is a local failure of the `socket()` call, and it is not the same as the on-the-wire fallback that downgrades an

MPTCP connection to plain TCP when the peer does not support it. An application that is hard-wired to request MPTCP, or a service wrapped with `mptcpize`, does not fall back gracefully: its socket creation fails. So the order is always check first (Question 3), then disable. Where you are unsure, stage the change ring by ring and watch for service errors rather than applying it fleet-wide at once.

The container and namespace caveat

Because `net.mptcp.enabled` is per network namespace, setting it in the host root namespace does not necessarily govern containers or other network namespaces on the same host. If your exposure model includes container workloads, apply the setting where those namespaces are created, or bake it into the base image or the orchestrator's node configuration, and verify inside a representative container rather than assuming inheritance.

What disabling does, and does not, fix

Disabling MPTCP removes the MPTCP code path from an attacker's reach. That closes the MPTCP-native flaws (the use-after-free, leak and exhaustion class in Part 2) on that host, and it removes the amplification path, so a bug like CVE-2026-98432 drops from its reported remote-code-execution worst case back to its plain-TCP outcome. What it does not do is patch the underlying plain-TCP bug where one exists: CVE-2026-98432's crash-and-corruption path on plain IPv4 TCP remained reachable with MPTCP off, and only the kernel patch closed it. Disabling MPTCP is therefore a reduction in reachability and severity, a standing control that shrinks the surface, not a substitute for patching the kernel.

Build-time removal

For appliances, minimal images, or any build where MPTCP is known never to be needed, the strongest form of the control is to compile the kernel without `CONFIG_MPTCP` at all, so the surface does not exist to be re-enabled. This is a kernel-build decision rather than a runtime one and is usually reserved for images you control end to end; for general-purpose hosts the sysctl setting, carried in configuration management, achieves the operational goal with far less effort.

Part 5: the vulnerability-management lens

MPTCP is a clean example of a kind of risk that standard vulnerability management handles poorly, because the risk is not a CVE. It is a configuration property of the asset that changes what future CVEs will be worth.

- **The signal lives in asset context, not in the score.** CVSS, EPSS and KEV describe a vulnerability. They cannot tell you that a given host has a feature enabled but idle, or that enabling it would upgrade a crash to a compromise. That fact comes from inventory (the three questions of Part 3), and it changes the real risk more than the base score does. This is the same argument the Vulnerability Prioritization Reference makes about carrying exposure alongside severity.
- **Model it as a control, not a patch.** "MPTCP disabled where unused" is a standing hardening control, like disabling an unused service or removing an unneeded kernel module. In an exposure or CTEM-style model it belongs on the control side of the ledger: it reduces reachability for a class of findings rather than remediating any one of them.
- **It pre-empts the next CVE, not just the last one.** The distinctive value of a surface-reduction control is that it is in place before the vulnerability is disclosed. A fleet that already runs with MPTCP disabled where

it is unused does not have an emergency when the next MPTCP use-after-free lands; those hosts are already out of reach, and the response narrows to the small set that genuinely use the feature. That is attack-surface reduction doing the work that reactive patching otherwise has to do under time pressure.

- **Severity numbers do not see reachability.** The MPTCP CVE history is full of this: a 9.8 that is "very unlikely to be reachable from the internet," a use-after-free that is "reachable via MPTCP over IPv6 but with no known attack scenario." Your own exposure model, host by host, is what resolves those into a queue, not the aggregator's headline figure.

Cross-cutting themes

- **A default-on feature is debt until it is proven useful.** MPTCP shipped enabled because the performance cost of leaving it on is near zero. The security cost is not zero, and it is paid by every host that carries the feature without using it. The discipline this argues for is general: inventory what is enabled, separate it from what is used, and remove the difference.
- **Exposure is three states, not a boolean.** Compiled, enabled, and in use are different questions with different answers and different consequences. Collapsing them into "is it on" is what lets a removable, idle surface sit unexamined across a fleet for years.
- **Attack-surface reduction compounds.** One control, applied once and carried in configuration management, defends against an open-ended series of future flaws in the same code. Patching defends against the one flaw that was disclosed. Both are necessary; the first is the cheaper of the two per unit of risk removed, and it is the one that works before disclosure.
- **A feature that gives attackers more control over memory is an amplifier.** MPTCP's relevance to CVE-2026-98432 was not a bug in MPTCP. It was that the feature's machinery made a separate bug more exploitable. When you weigh an optional subsystem, weigh not only its own defects but what it does to the severity of defects elsewhere. See the CVE-2026-98432 case study.
- **The cheapest finding to close is the one you closed before it was filed.** The whole value of knowing your MPTCP exposure and removing it where it is idle is that the work is done on your schedule, in advance, rather than during the next critical-kernel-CVE fire drill.

Sources

- The Linux kernel networking documentation for MPTCP (`Documentation/networking/mptcp.rst`), for the opt-in design, the `IPPROTO_MPTCP` protocol number 262, the `socket()` errno values (`EINVAL` , `EPROTONOSUPPORT` , `ENOPROTOOPT`), the fallback-to-plain-TCP behaviour, the plain accepted-socket behaviour of MPTCP listen sockets, and the force-enable techniques (`mptcpize` via LD_PRELOAD, `mptcpify` via eBPF, SystemTAP).
- The Linux kernel MPTCP sysctl documentation (`Documentation/networking/mptcp-sysctl.rst`), for the `net.mptcp.enabled` boolean, its default of 1, its per-namespace scope, and the other MPTCP sysctl knobs and their defaults.
- RFC 8684 (Multipath TCP version 1) and the earlier RFC 6824 (version 0), for the protocol's purpose and design.

- Red Hat's vulnerability page for CVE-2026-31669, for the use-after-free in IPv6 subflow child sockets, the "at least a denial of service via kernel crash" and "memory corruption class" impact statements, the CVSS 7.5 rating against cve.org's 9.8, and the statements that there is no known attack scenario and no confirmed practical trigger under default configurations.
- Kernel stable-tree advisories and distribution vendor trackers for the other MPTCP flaws cited (CVE-2026-80588 receive-path memory growth, CVE-2025-40258 race-condition use-after-free, CVE-2022-50071 subflow resource leak with its `net.mptcp.enabled=0` workaround, and CVE-2021-47152 data-stream corruption); third-party aggregators were found to disagree on attack vector and severity, so the authoritative advisory governs in each case.
- The companion TeamStarWolf case study on CVE-2026-98432, for the IPv4 TCP use-after-free, its dependence on an MPTCP listener for the reported remote-code-execution outcome versus denial of service otherwise, and the grading of that exploit claim as Reported.

Related references

- Linux Hardening Reference
- Network Protocols Security
- Network Attacks Reference
- Vulnerability Prioritization Reference
- Vulnerability Management Reference
- The CVE-2026-98432 Linux Kernel Flaw and an AI-Written Exploit
- Notable Incidents